[L5] Containerized Model Serving

Start Assignment

Due Wednesday by 11:59pm **Points** 50 **Submitting** a text entry box or a file upload
File Types zip, py, and txt

[L5] Containerized Model Serving

In this lab you will use Docker Model Runner to serve a pre-built model, then containerize one of your own models from Week 3 or 4 into a dedicated serving container with a FastAPI endpoint. By the end, your app will have a full multi-service architecture: frontend, API, and model service.

Overview

Total Points	50
Submission	ZIP file containing your project, OR a .txt file with your GitHub repo link
Key Technologies	Docker, Docker Model Runner, FastAPI, PyTorch or TensorFlow

Part 1: Docker Model Runner (15 points)

Get hands-on with Docker Model Runner — Docker's built-in support for running AI models as containers.

Requirements:

- **Pull a pre-built model** using Docker Model Runner:

```
docker model pull ai/smollm2:360M-instruct-q4_K_M
```

This pulls a small language model optimized for local inference. You can also explore other available models.

- **Run the model** and send a prediction request:

```
docker model run ai/smollm2:360M-instruct-q4_K_M
```

- **Send a request** to the model's inference endpoint using curl or Python requests:

```
# Using curl (OpenAI-compatible API)
curl http://localhost:12434/engines/llama.cpp/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "ai/smollm2:360M-instruct-q4_K_M",
    "messages": [{"role": "user", "content": "Explain what Docker is in one sentence."}]
  }'
```

```
# Or using Python requests
import requests

response = requests.post(
    "http://localhost:12434/engines/llama.cpp/v1/chat/completions",
    json={
        "model": "ai/smollm2:360M-instruct-q4_K_M",
        "messages": [{"role": "user", "content": "Explain what Docker is in one sentence."}]
    }
)
print(response.json())
```

- **Document in your README:**
  - What model image did you pull?
  - What endpoint does it expose?
  - What was the response to your test query?

Key concept: Docker Model Runner gives you a standardized way to run AI models without writing Dockerfiles or configuring serving infrastructure. The model runs as a container and exposes an OpenAI-compatible HTTP API.

Part 2: Containerize Your Own Model (20 points)

Take one of your models from Week 3 (PyTorch) or Week 4 (TensorFlow) and build a dedicated model-serving container.

Step 1: Create the Model Service

Create a new directory (e.g., `model_service/`) with:

```
model_service/
  Dockerfile
  serve.py            # FastAPI app that loads and serves your model
  requirements.txt    # Dependencies (fastapi, uvicorn, torch or tensorflow)
  model/              # Your saved model file (.pt, .keras, or .onnx)
```

Step 2: Build the FastAPI Server

```
from fastapi import FastAPI
import torch  # or tensorflow
import numpy as np

app = FastAPI()

# Load model at startup
model = None

@app.on_event("startup")
def load_model():
    global model
    model = torch.load("model/my_model.pt")  # or keras.models.load_model(...)
    model.eval()

@app.get("/health")
def health():
    return {"status": "healthy", "model_loaded": model is not None}

@app.post("/predict")
def predict(data: dict):
    # Convert input to tensor, run prediction, return result
    input_data = np.array(data["features"])
    # ... run model prediction ...
    return {"prediction": result, "model": "your-model-v1"}
```

Step 3: Write the Dockerfile

```
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8001
CMD ["uvicorn", "serve:app", "--host", "0.0.0.0", "--port", "8001"]
```

Step 4: Add to docker-compose.yml

```
services:
  api:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      model-service:
        condition: service_healthy

  model-service:
    build: ./model_service
    ports:
      - "8001:8001"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8001/health"]
      interval: 10s
      timeout: 5s
      retries: 3
```

Step 5: Wire the API to the Model Service

Update your main API so it **calls the model service** instead of loading the model directly:

```
import requests

@app.post("/predict")
def predict(data: PredictionRequest):
    # Forward to model service instead of running locally
    response = requests.post(
        "http://model-service:8001/predict",
        json={"features": data.features}
    )
    return response.json()
```

Verify:

```
docker-compose up --build
# Then test the full flow:
curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d '{"features": [5.1, 3.5, 1.4, 0.2]}'
```

What changed: Your main API no longer imports PyTorch or TensorFlow. It just makes an HTTP call to the model service. This means you can scale, update, or swap the model independently of the API.

Part 3: Multi-Model Architecture (15 points)

Bring everything together into a full multi-service architecture.

Requirements:

- **docker-compose.yml** with all services:
  - `frontend` — serves the UI (if you have one from previous weeks)
  - `api` — your main FastAPI application
  - `model-service` — your containerized model
- `/predict` **endpoint** on the main API that proxies to the model service — the client never talks directly to the model container
- **Update your README** with an architecture diagram showing the request flow:

```
Client → API (:8000) → Model Service (:8001) → prediction
                  ↓
            /health checks
```

- **Screenshot** showing the full stack running with a prediction flowing through all services (docker-compose logs or curl output showing the request chain)
- **At least 3 commits this week** showing incremental progress (not one big commit at the end)

Grading Rubric

Criteria	Points	Details
Docker Model Runner: pull & run	5	Successfully pulled and started a model with Docker Model Runner
Inference request sent & documented	5	Sent request via curl or Python, documented image, endpoint, and response
Model Runner exploration documented	5	README explains what was pulled and how it works
Dockerfile for model service	5	Working Dockerfile that loads saved model and installs dependencies
FastAPI /predict endpoint on model service	5	Accepts input, runs model, returns prediction as JSON
/health endpoint on model service	3	Returns health status, used by docker-compose healthcheck
API proxies to model service	7	Main API calls model service over HTTP; no direct model import in API
docker-compose.yml with all services	5	Frontend, API, and model-service defined; healthchecks configured
Architecture diagram in README	5	Shows request flow through all services
Full stack screenshot	5	Shows docker-compose running with a prediction flowing through all services
At least 3 commits	5	Incremental progress visible in git history
Total	55	*Scaled to 50 points*

Tips

- **Start with Part 1** — Docker Model Runner is the fastest way to see a model running in a container. Get it working before you build your own.
- **Use a small model** for Part 2 — your `iris/make_moons` model from Week 3 is perfect. Don't over-engineer it.
- **Test the model service standalone first** before wiring it into docker-compose. Run it directly with `uvicorn serve:app --port 8001` and verify `/health` and `/predict` work.
- If your model service takes a while to start, the **healthcheck + depends_on** in docker-compose ensures the API waits.
- **Commit early and often** — don't save all 3 commits for the last hour.
- Docker Model Runner docs: https://docs.docker.com/desktop/features/model-runner/

Week 6: LLM Integration — adding ChatGPT-like capabilities to your app via APIs.

[L5] Containerized Model Serving

Criteria	Ratings				Pts
Docker Model Runner Pre-built model pulled, inference endpoint queried successfully, documented	15 pts Complete and documented	10 pts Works but weak docs	5 pts Partial	0 pts Missing	15 pts
Custom Model Container Dockerfile loads saved model, FastAPI endpoint serves predictions, /health endpoint works	12 pts Complete with health check	8 pts Serves predictions, no health	4 pts Partial	0 pts Missing	12 pts
Docker Compose Integration Model service added to docker-compose, main API proxies to model service	8 pts Full integration works	5 pts In compose but proxy issues	2 pts Partial	0 pts Missing	8 pts
End-to-End Prediction Flow Prediction flows from frontend through API to model service and back	5 pts Works end to end	3 pts Partial flow		0 pts Missing	5 pts
Architecture Diagram & README Updated README with new architecture, diagram showing all services	5 pts Complete	3 pts Partial		0 pts Missing	5 pts
Screenshot & Commits Screenshot of full stack running, 3+ meaningful commits	5 pts Both present	3 pts One missing		0 pts Missing	5 pts

Total Points: 50